

AWS Certified Generative AI Developer — Professional

The Use Case

- Insurance company modernising claims assessment
- Data sources:** claim forms (tabular), damage photos (image), customer call recordings (audio), historical claim data (tabular)
- Goals:** assess claim validity, estimate repair cost, detect fraud, give adjusters insight
- Scale:** 5,000+ claims per day

How The Three Tasks Differ

Task	The question it answers
1.1	How do I build the system?
1.2	How do I select and manage the model?
1.3	What happens to the data before it ever reaches the model?

What is new here: multimodal. The first two tasks handled text only. Here four data types are processed in parallel, then fused.

```
Data Sources -> Validation -> Processing -> Data Fusion
              -> Format -> Enhance -> Bedrock -> Outputs
```

Note that Bedrock sits at the **end**. Five stages precede it.

1. Data Validation

Garbage in, garbage out — and in GenAI this is dangerous because a bad answer still *reads* convincingly. A wrong policy number leads to wrong coverage, a wrong claim decision, and lost money.

Four tools, four different jobs

These are not competitors. Each catches something the others cannot.

AWS Glue Data Quality — errors you can anticipate

- Rule-based validation for **structured** data, written in **DQDL** (Data Quality Definition Language)
- Example rules: policy number matches a pattern; claim amount within policy limits; damage date inside the coverage period; customer ID not null
- Can generate **rule recommendations** by profiling the data — useful when starting out
- Use when you already know what "correct" looks like.** Automated, runs continuously in production.

SageMaker Data Wrangler — errors you cannot anticipate

- Visual, interactive exploration of distributions, outliers and anomalies
- Transcript example: revealed that **policy number formatting was inconsistent across legacy systems**
- No rule would have caught that — you didn't know the formats differed until you saw the chart
- Exploration tool** (run once, human-driven), not a production pipeline

Tool	Catches
Glue Data Quality	Known unknowns — you know what can go wrong
Data Wrangler	Unknown unknowns — you didn't know to look

Custom Lambda — what no off-the-shelf tool can do

- VIN matches the reported make and model (needs a vehicle database)
- Medical procedure codes are valid
- Property damage description aligns with the photographic evidence (needs both text and image)

This is **domain knowledge, not format checking** — which is why it has to be written as code.

CloudWatch — quality over time

- The other three check a batch; CloudWatch watches the **trend**
- Validation pass rate drifting 94% → 91% → 87% signals an upstream change nobody announced
- Alarms on thresholds; dashboards for visibility

Validation by data type

Type	What to check
Tabular	Schema, value ranges, nulls, relationships
Text	Language detection, coherence, encoding, length
Image	Resolution, format, corruption, content verification (Rekognition)
Audio	Duration, sample rate, quality, silence

A blurry damage photo should be **rejected before the model sees it** — otherwise the model will invent something.

Feedback loop

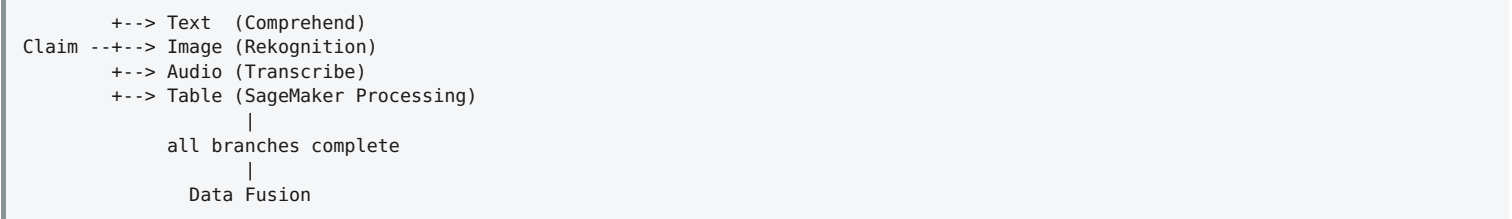
```
Data validated -> passed -> model produced a poor answer
-> go back: what did validation miss?
-> add a new rule
```

- Failed data goes to **human review**, never silently discarded
- Validation rules evolve.** Writing them once and forgetting is the mistake.

2. Data Processing

Four data types, all different, all needing separate treatment before they can be combined. Running them sequentially would be far too slow — hence parallel.

AWS Step Functions — the orchestrator



Why Step Functions rather than Lambda:

- Lambda caps at 15 minutes; Transcribe on a long call can exceed that
- Per-branch retry and error handling
- **Waiting for all branches** is built in
- Visual execution history shows exactly where a run stalled

This is a **different use of Step Functions** from Task 1.2, where it implemented the circuit breaker. Here it is parallel orchestration.

Text — Amazon Comprehend

- Managed NLP, no model invocation — cheaper and faster than calling an FM
- Language detection, entity extraction, sentiment (anger can be a fraud signal), key phrases, PII detection
- Comprehend Medical exists for clinical text
- Plus **custom Lambda** for insurance-specific normalisation

Image — Rekognition + SageMaker Processing

Service	Role
Rekognition	Generic managed vision — object detection, image quality, text in image (number plates), moderation
SageMaker Processing	Custom work a generic service can't do — damage severity assessment , resizing, cropping, normalising to the model's input spec

The generic-vs-custom split is an exam-relevant distinction.

Audio — Amazon Transcribe

- **Transcribe** — speech to text
- **Transcribe Call Analytics** — purpose-built for contact-centre audio: sentiment, talk time, interruptions, **speaker diarization** (who is speaking)
- Transcript notes this surfaces **fraud indicators**
- An hour-long recording can't go to the model directly — transcribe, then summarise, then send

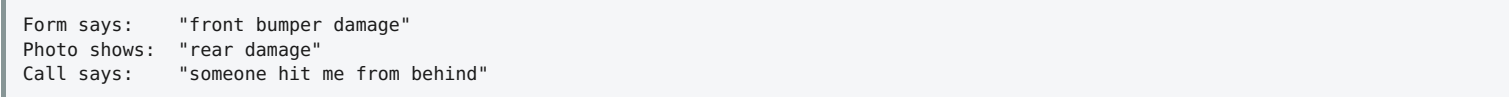
Tabular — SageMaker Processing

- Feature engineering, normalisation, embedding generation
- Runs on large datasets, outside Lambda's constraints

Data Fusion — the hardest part

Not simple concatenation. Two problems:

1. **Alignment** — at 3:45 the caller says "they hit me from behind"; photo #4 shows a damaged rear bumper. These refer to the same fact and must be linked.
2. **Contradiction detection** — where the real value is



The form is wrong — or it's fraud. **This is only visible when the modalities sit side by side.** Examined separately it is invisible. This is why fraud detection improved 43%.

Fusion output — one clean structure containing the structured fields, a text summary, image findings, a call summary, and any cross-modal flags.

3. Formatting Input Data For FM Inference

The three inference parameters

Parameter	What it controls
temperature	Randomness in token selection. 0.0 = deterministic, 1.0 = creative.
top_p	Nucleus sampling — how wide a pool of candidate tokens is considered. 0.1 = only the most likely.
max_tokens	Length cap on the output only, not the input.

- **Insurance needs 0.0-0.2.** The same claim must not produce a different assessment each time.
- Rule: extraction, classification, structured output → **low**. Creative writing, brainstorming → higher.
- **Tune temperature or top_p, not both** — changing both makes the effect impossible to attribute.
- `max_tokens` too low truncates mid-answer; too high can raise cost and latency

Dynamic prompt construction

One fixed prompt cannot serve every claim — auto and property claims need different considerations, and a suspected-fraud claim needs different framing again.

```
system instructions
+ claim-type-specific rules
+ fused data
+ enrichment (vehicle specs, repair benchmarks)
+ the question
```

Transcript reports this significantly improved both relevance and accuracy. **Link to Task 1.1:** Prompt Management stores the versioned templates; this is where they get populated.

Conversation history management

Adjusters ask follow-ups ("summarise this claim" → "what's the repair cost?" → "any red flags?"). Without history, the third question is meaningless.

The model remembers nothing. The entire history is re-sent with every request. "Memory" is something you build.

Technique	How it works
Sliding window	Keep only the last N messages
Summarisation	Compress older turns into a summary
Hybrid	Summary of old turns + recent turns in full — most common in practice

Multimodal payloads

- Images are **base64-encoded**, which inflates size by roughly 33%
- Each model has its own limits: image count, file size, supported formats
- Excessive resolution gets downscaled by the model anyway — wasted tokens
- **Audio cannot be sent directly** — it must be transcribed to text first

This is precisely why the processing pipeline exists: raw data cannot simply be handed to the model.

SageMaker endpoints — a different formatting problem

	Format
Bedrock	Managed API, standard JSON payloads
SageMaker endpoint	Your model, your format — serialization (JSON/CSV/protobuf), batching strategy, input tensor shape per model architecture

4. Enhancing Input Data Quality

Purpose	
Validation	Blocks bad data
Enhancement	Improves data that already passed

The starting point is analysing FM performance patterns: when the model answers poorly, trace back to the input. Transcript examples — inconsistently formatted damage descriptions, and incomplete accident narratives. **Both passed validation.** No rule was broken; the input was simply poor for the model.

Text reformatting using Bedrock itself

```
raw text -> small/cheap model (cleanup) -> clean text -> main model
```

- Expands insurance abbreviations (TL → Total Loss, PD → Property Damage)
- Corrects grammar, standardises inconsistent formats
- The main model then spends its capacity on assessment rather than decoding shorthand
- **Cost consideration:** this is an extra invocation. Use a small model, and only where inputs are genuinely messy.

Entity standardisation with Comprehend

```
"3rd Jan 2025", "01/03/2025", "Jan 3, 2025"  ->  2025-01-03
```

The model should see **one shape**, not three. Note this is a *different use* of Comprehend from the processing stage — there it extracted information, here it normalises it.

Custom Lambda — domain normalisation

- **Terminology:** bumper / fender / front end → standard part names
- **Unit conversion:** miles ↔ km, currency
- **Format harmonisation:** VIN, policy numbers, phone numbers

No off-the-shelf service knows your company's internal shorthand.

Noise reduction

- HTML tags, PDF artefacts, repeated whitespace, odd line breaks, encoding errors
- Boilerplate: footers, disclaimers, signatures
- **Why it matters:** all of it consumes tokens and dilutes attention. Removing 30% noise saves 30% cost and improves the answer.

Content enrichment — the highest-value step

Everything above cleans. Enrichment **adds**.

```
Claim:  "2019 Honda Civic, rear bumper damage"  
      + vehicle specifications (part numbers)  
      + regional repair cost benchmark (avg $850)  
      + claim history for this model
```

Now the model can say: "*the \$4,500 estimate is 5× the benchmark — flag it.*" **Without enrichment it cannot**, because it has no idea what normal looks like.

This is **RAG** — the same pattern as Task 1.1. The difference is that the facts come from reference databases rather than document stores.

Measuring the input-output relationship

```
Enrichment on  -> accuracy 97%  
Enrichment off -> accuracy 89%
```

Without this measurement you are guessing which enhancement steps earn their cost and which are waste.

The Complete Pipeline

Raw data	
-> Validation	(block the bad)
-> Processing	(per-type, parallel, then fusion)
-> Formatting	(JSON payload, dynamic prompt)
-> Enhancement	(clean + enrich)
-> Bedrock	

Five stages before the model. That is what makes Task 1.3 entirely a "before the model" domain.

Service Discriminations

Choice	Pick when
Glue Data Quality	Rule-based checks on structured data, automated and recurring
SageMaker Data Wrangler	Visual exploration to discover unknown data issues
Custom Lambda validation	Domain logic no managed service covers
Step Functions	Parallel branches, long-running steps, join/wait, per-branch retry
Comprehend	Managed NLP — entities, sentiment, language, PII. Cheaper than an FM call.
Rekognition	Generic managed vision
SageMaker Processing	Custom transformation at scale, beyond Lambda limits
Transcribe Call Analytics	Contact-centre audio needing diarization and sentiment

Reported Outcomes

- 5,000+ claims per day, **97% accuracy** in assessment and cost estimation
- **78% reduction** in claims returned for additional information
- **65% reduction** in processing time — straightforward claims assessed in minutes rather than days
- **43% improvement** in fraud detection, roughly \$12M estimated annual saving

Service limits and feature availability change. Verify specific numbers against current AWS documentation before the exam.